Wireshark Filters Analysis

Overview

Wireshark Filters: Wireshark filters allow filtering packets based on protocols (HTTP, TLS, TCP, DNS, SSDP, etc.), specific fields (such as TCP flags), and logical operators (`and`, `or`, `!` for negation).

Involved Protocols:

- **HTTP**: Application layer protocol for hypertext transfer (web requests).
- **TLS**: Transport layer protocol for secure connections (handshake type 1 is the "Client Hello" message).
- **TCP**: Transport protocol, where SYN and ACK flags are used in connection establishment (three-way handshake).
- **DNS**: Protocol for domain name resolution.
- **SSDP**: Simple Service Discovery Protocol used for device discovery on local networks (used in UPnP).

Operators:

- `or`: Shows packets that meet at least one of the conditions.
- `and`: Requires both conditions to be true.
- `!`: Negates a condition (excludes packets).
- `eq`: Compares equality (for example, `tls.handshake.type eq 1` means TLS handshake type equals 1).

Filter 1: `(http.request or tls.handshake.type eq 1) and !(ssdp)`

What It Filters

This filter selects packets that meet one of the following conditions, but excludes SSDP packets:

1. **HTTP Requests**: Packets containing an HTTP request (e.g., GET, POST).
2. **TLS Client Hello**: TLS packets with a handshake message of type 1 (start of a secure connection, such as HTTPS).

Exclusion: Does not include SSDP protocol packets.

Breakdown

- `http.request`: Filters packets that are HTTP requests (not responses). This includes any request sent by a client to a web server (e.g., when loading a page).

- `tls.handshake.type eq 1`: Filters TLS packets where the handshake message type is 1, corresponding to the "Client Hello" message. This is the first step in establishing a TLS/SSL connection (e.g., when initiating an HTTPS session).
- `or`: Shows packets that meet at least one of the above conditions (HTTP or TLS Client Hello).
- `and !(ssdp)`: Excludes any packet using the SSDP protocol, which is typically device discovery traffic on local networks (such as printers or IoT devices).

Practical Use

This filter is useful for analyzing web traffic (HTTP and HTTPS) while ignoring local network discovery traffic (SSDP). For example, you could use it to:

- Monitor web requests on a network.
- Analyze secure connection initiation (HTTPS) without contaminating results with SSDP traffic.

Filter 2: `(http.request or tls.handshake.type eq 1 or (tcp.flags.syn eq 1 and tcp.flags.ack eq 0)) and !(ssdp)`

What It Filters

This filter extends the previous one by including a third condition: TCP packets that initiate a connection (SYN without ACK). It continues to exclude SSDP.

1. **HTTP Requests**: Same as before, HTTP requests.
2. **TLS Client Hello**: Same as before, TLS "Client Hello" messages.
3. **TCP SYN**: TCP packets with SYN flag set (1) and ACK flag unset (0), indicating the start of a TCP connection (first step of the three-way handshake).
4. **Exclusion**: Does not include SSDP packets.

Breakdown

- `http.request`: Same as in filter 1.
- `tls.handshake.type eq 1`: Same as in filter 1.
- `(tcp.flags.syn eq 1 and tcp.flags.ack eq 0)`: Filters TCP packets that have the SYN (synchronization) flag set and the ACK (acknowledgment) flag unset. This corresponds to the first packet in establishing a TCP connection (the "SYN" of the three-way handshake: SYN → SYN-ACK → ACK).
- `or`: Shows packets that meet at least one of the three conditions (HTTP, TLS Client Hello, or TCP SYN).
- `and !(ssdp)`: Excludes SSDP packets.

Practical Use

This filter is broader than the first one, as it includes not only HTTP and TLS traffic but also any attempt to initiate a TCP connection. It's useful for:

- Analyzing network connection initiation (TCP) in addition to web traffic.
- Identifying connection patterns (e.g., port scans or failed connection attempts).
- Excluding SSDP traffic to focus on application-specific traffic or connections.

Filter 3: `(http.request or tls.handshake.type eq 1 or (tcp.flags.syn eq 1 and tcp.flags.ack eq 0) or dns) and !(ssdp)`

What It Filters

This filter is the most comprehensive of the three, as it adds DNS packets to the previous conditions while continuing to exclude SSDP.

1. **HTTP Requests**: HTTP requests.
2. **TLS Client Hello**: TLS "Client Hello" messages.
3. **TCP SYN**: TCP packets with SYN=1 and ACK=0 (TCP connection initiation).
4. **DNS**: Any DNS packet (queries or responses).
5. **Exclusion**: Does not include SSDP packets.

Breakdown

- `http.request`: Same as before.
- `tls.handshake.type eq 1`: Same as before.
- `(tcp.flags.syn eq 1 and tcp.flags.ack eq 0)`: Same as in filter 2.
- `dns`: Filters any DNS packet, including queries (asking for domain resolution, such as A or AAAA records) and responses (providing the IP address or other information).
- `or`: Shows packets that meet at least one of the four conditions (HTTP, TLS Client Hello, TCP SYN, or DNS).
- `and !(ssdp)`: Excludes SSDP packets.

Practical Use

This filter is ideal for capturing a more complete view of network traffic related to:

- Web activity (HTTP and HTTPS).

- TCP connection initiation.
- Domain name resolution (DNS).

It's useful in scenarios such as:

- Debugging web connectivity issues (e.g., checking if DNS queries fail before a TCP or HTTP connection attempt).
- Monitoring critical network traffic while excluding irrelevant protocols like SSDP.

Filter Comparison

Filter	HTTP	TLS Client Hello	TCP SYN	DNS	Excludes SSDP
1	✓	✓			✓
2	✓	✓	✓		✓
3	✓	✓	✓	✓	✓

Each filter becomes progressively more inclusive:

- **Filter 1** focuses on web traffic (HTTP and HTTPS).
- **Filter 2** adds TCP connection initiation.
- **Filter 3** also includes DNS traffic, covering name resolution.

Additional Notes

1. **Why exclude SSDP?**: SSDP generates a lot of traffic on local networks due to device discovery (such as in home networks with IoT devices). Excluding it reduces "noise" in the capture, allowing focus on relevant traffic.
2. **Example Scenario**: If you're debugging why a web page won't load, **Filter 3** allows you to see:
   - DNS queries (Is the domain being resolved?).
   - TCP connection attempts (Is the connection being initiated?).
   - HTTP requests or TLS initiation (Does the request reach the server?).
3. **Performance**: Broader filters (like filter 3) may capture more packets, which could slow down analysis in large captures. Adjust the filter according to your use case.
4. **Verification in Wireshark**:
   - Apply these filters in the Wireshark filter bar.
   - Use the packet view to inspect details (e.g., DNS query content or HTTP headers).